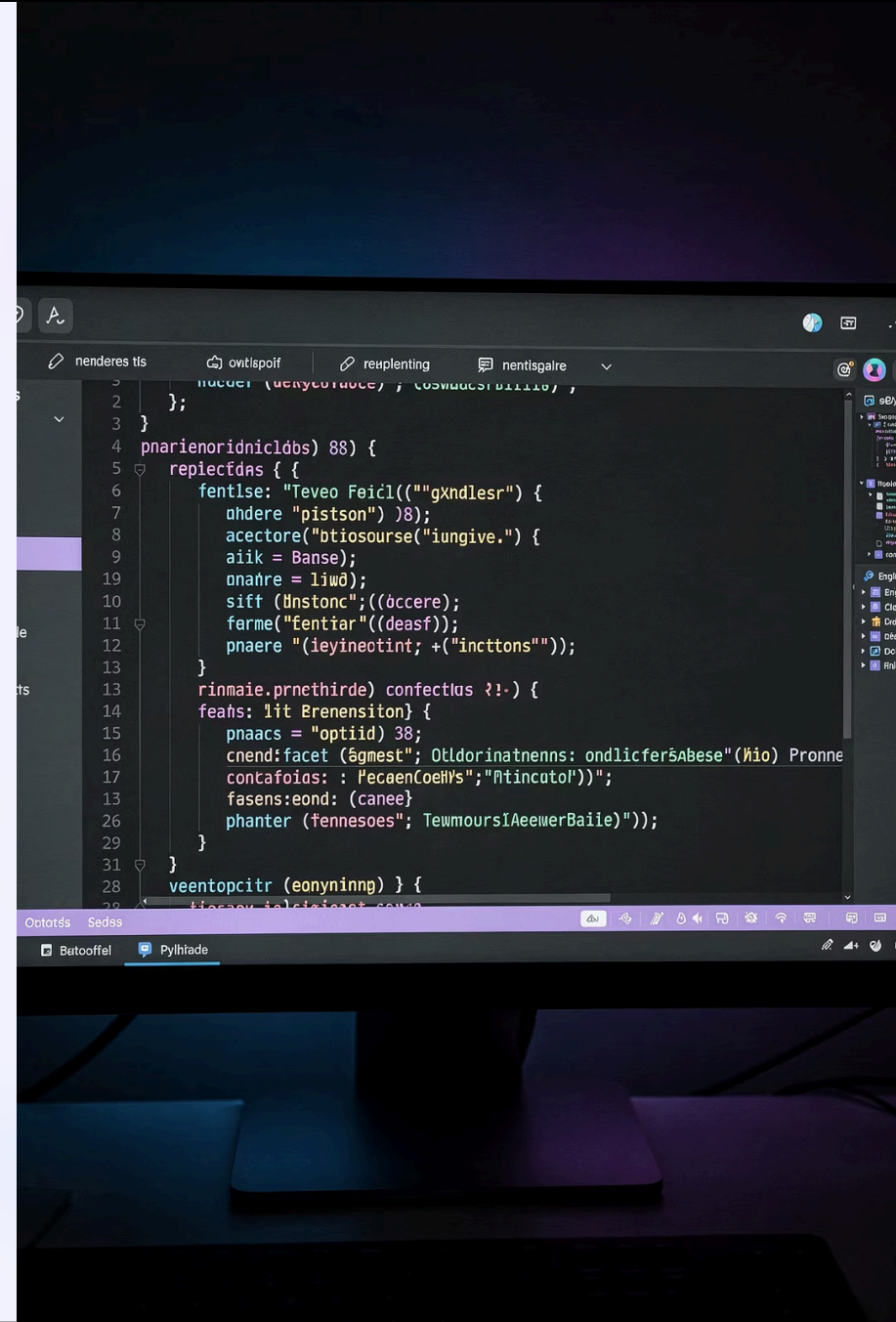


# Funções em Python

ALGORITMOS II

MODULARIZAÇÃO DE CÓDIGO

Nesta aula, vamos explorar um dos conceitos mais fundamentais da programação: as **funções**. Aprenderemos como criar blocos de código reutilizáveis, organizar nossa lógica em módulos coesos e escrever programas mais limpos, legíveis e fáceis de manter em Python.



# Objetivo da Aula

Ao final desta aula, você será capaz de criar, chamar e reutilizar funções em Python, compreendendo todos os seus componentes essenciais.



## Definir Funções

Aprender a declarar funções usando a palavra-chave `def`, com ou sem parâmetros, de forma correta e idiomática em Python.



## Chamar Funções

Entender como invocar funções, passar argumentos e capturar valores de retorno em diferentes contextos de um programa.



## Reutilizar Código

Aplicar o princípio DRY (*Don't Repeat Yourself*) para evitar duplicação de lógica e tornar o código mais modular e organizado.



## Boas Práticas

Adotar convenções de nomenclatura, tamanho ideal de funções e uso correto de escopo para escrever código profissional e legível.

# O que é uma Função?

Uma **função** é um bloco de código nomeado, reutilizável, que executa uma tarefa específica. Em vez de repetir o mesmo código em vários lugares do programa, você o encapsula em uma função e a chama sempre que precisar daquele comportamento.

## Por que usar funções?

- **Reutilização:** escreva uma vez, use várias vezes
- **Legibilidade:** nomes descritivos explicam a intenção
- **Manutenção:** corrija em um único lugar
- **Testabilidade:** isole e teste partes do código
- **Abstração:** esconda detalhes de implementação

## Anatomia de uma função

Toda função em Python possui:

- Um **nome** que identifica o que ela faz
- **Parâmetros** opcionais (entradas)
- Um **corpo** com a lógica a ser executada
- Um **valor de retorno** opcional (saída)

Funções são os blocos fundamentais da **modularização**, que divide um programa grande em partes menores e mais gerenciáveis.

# Sintaxe de uma Função em Python

A definição de uma função em Python segue uma estrutura bem definida. Veja os elementos obrigatórios e opcionais:

```
def nome_funcao(parametro1, parametro2):  
    """Docstring: descrição opcional da função."""  
    # corpo da função — lógica aqui  
    resultado = parametro1 + parametro2  
    return resultado # retorna o valor (opcional)
```

1

## Palavra-chave def

Inicia a definição de uma função. É obrigatória e indica ao Python que o bloco a seguir é uma função nomeada.

2

## Nome da Função

Deve ser descritivo e seguir o padrão **snake\_case** (letras minúsculas separadas por underscores), conforme a PEP 8 do Python.

3

## Parâmetros

Variáveis listadas entre parênteses que recebem valores no momento da chamada. Uma função pode ter zero ou mais parâmetros.

4

## Dois-pontos e Indentação

Os `:` encerram a linha de definição. O corpo da função **deve ser indentado** com 4 espaços — isso é obrigatório em Python.

5

## Instrução return

Encerra a execução e devolve um valor ao chamador. Se omitida, a função retorna `None` automaticamente.

# Exemplo Simples: Função sem Parâmetros

Vamos começar com o exemplo mais básico possível: uma função que não recebe nenhum dado de entrada e apenas exibe uma mensagem na tela.

## Definição e Chamada

```
def saudacao():  
    print("Olá!")  
  
# Chamando a função  
saudacao()  
saudacao()  
saudacao()
```

## Saída:

```
Olá!  
Olá!  
Olá!
```

## O que acontece aqui?

Ao escrever `saudacao()`, o Python localiza a definição da função e executa todas as instruções dentro do seu corpo. O grande benefício é que:

- A lógica é definida **uma única vez**
- Pode ser executada **quantas vezes quiser**
- Se precisar mudar a mensagem, altera-se **em um só lugar**



Perceba os parênteses `()` na chamada: eles são obrigatórios mesmo quando não há argumentos. Escrever apenas `saudacao` sem parênteses não executa a função — apenas referencia o objeto função.

# Função com Parâmetro

Parâmetros tornam as funções muito mais poderosas e versáteis. Em vez de operar sempre sobre os mesmos valores, a função recebe dados externos e os processa dinamicamente.

```
def dobro(x):  
    return x * 2
```

# Usando a função com diferentes valores

```
print(dobro(5)) # Saída: 10
```

```
print(dobro(3)) # Saída: 6
```

```
print(dobro(7.5)) # Saída: 15.0
```

## Parâmetro vs. Argumento

O **parâmetro** é a variável na definição da função – no exemplo, `x`. O **argumento** é o valor real passado na chamada – como `5`, `3` ou `7.5`. Os dois termos são frequentemente confundidos, mas têm significados distintos.

## Tipagem Dinâmica

Python não exige que você declare o tipo do parâmetro. A função `dobro` funciona com inteiros, floats e até strings (que seriam duplicadas). Isso é chamado de **duck typing**: o comportamento depende do tipo do argumento passado em tempo de execução.

## Múltiplos Parâmetros

Uma função pode receber quantos parâmetros forem necessários, separados por vírgulas: `def calcular(a, b, c):`. Python também suporta parâmetros com **valores padrão**, como `def dobro(x, fator=2):`, que se tornam opcionais na chamada.

# Função com Retorno de Valor

A instrução `return` é o mecanismo pelo qual uma função **comunica seu resultado** de volta ao código que a chamou. Isso permite compor funções, armazenar resultados e usá-los em expressões.

## Exemplo: Soma de dois números

```
def soma(a, b):  
    return a + b
```

# O valor retornado pode ser:

# 1. Armazenado em variável

```
resultado = soma(10, 5)  
print(resultado)    # 15
```

# 2. Usado diretamente em expressão

```
print(soma(3, 4) * 2) # 14
```

# 3. Passado para outra função

```
print(soma(soma(1, 2), 3)) # 6
```

## Sobre o `return`

- Encerra a execução da função imediatamente
- Pode retornar qualquer tipo de dado
- Pode retornar **múltiplos valores** como tupla: `return a, b`
- Sem `return`, a função devolve `None`
- Pode haver múltiplos `return` em caminhos diferentes



Uma função que usa `print()` **não retorna** o valor – ela apenas o exibe. Para reutilizar o resultado em cálculos, sempre use `return`.

# Chamando uma Função

Definir uma função não a executa — ela precisa ser **chamada**. A chamada é feita pelo nome da função seguido de parênteses com os argumentos correspondentes aos parâmetros definidos.

```
def soma(a, b):  
    return a + b  
  
# Chamada e captura do resultado  
resultado = soma(2, 3)  
print(resultado)    # Saída: 5  
  
# Chamada direta dentro do print  
print(soma(10, 20)) # Saída: 30
```

1

## 1. Chamada

O programa encontra `soma(2, 3)` e interrompe o fluxo atual para executar a função.

2

## 2. Passagem de Argumentos

Os valores `2` e `3` são copiados para os parâmetros `a` e `b` dentro da função.

3

## 3. Execução

O corpo da função roda: `a + b` é calculado, resultando em `5`.

4

## 4. Retorno

O valor `5` é devolvido ao ponto de chamada e armazenado em `resultado`.



# Escopo de Variáveis

O **escopo** determina onde uma variável pode ser acessada no código. Em Python, variáveis criadas dentro de uma função são **locais** – existem apenas durante a execução daquela função e são destruídas quando ela termina.

## Variáveis Locais vs. Globais

```
x = 10 # variável GLOBAL

def minha_funcao():
    y = 20 # variável LOCAL
    print(x) # OK: acessa global
    print(y) # OK: acessa local

minha_funcao()
print(x) # OK: 10
print(y) # ERRO: y não existe aqui!
```

## Regras de Escopo (LEGB)

Python busca variáveis na ordem **LEGB**:

1. **Local** – dentro da função atual
2. **Enclosing** – funções externas (closures)
3. **Global** – nível do módulo
4. **Built-in** – nomes nativos do Python

Modificar uma variável global dentro de uma função requer a palavra-chave `global`, mas isso é considerado **má prática** – prefira sempre passar parâmetros e usar retorno.

# Boas Práticas ao Escrever Funções

Escrever funções de qualidade vai além de simplesmente fazer o código funcionar. Bons programadores seguem convenções e princípios que tornam o código mais legível, testável e fácil de manter a longo prazo.



## Funções Pequenas e Focadas

Cada função deve fazer **uma coisa só** e fazê-la bem. Se uma função precisa de muitos comentários para ser entendida, provavelmente está fazendo coisas demais. O ideal é que uma função caiba inteira na tela – em torno de 10 a 20 linhas. Divida funções grandes em funções menores colaborando entre si.



## Nomes Claros e Descritivos

O nome deve revelar a **intenção**. Prefira `calcular_media()` a `cm()`. Use verbos para funções que executam ações (`salvar_arquivo`, `enviar_email`) e substantivos para funções que retornam valores (`media_turma`). Siga o padrão **snake\_case** da PEP 8.



## Docstrings e Comentários

Use **docstrings** (strings entre `"""`) logo após o `def` para descrever o que a função faz, seus parâmetros e o valor de retorno. Isso é acessível via `help()` e ferramentas de documentação como Sphinx. Comente o *porquê*, não o *o quê* óbvio.



## Evite Efeitos Colaterais

Prefira funções **puras**: recebem entrada, produzem saída, sem modificar variáveis globais ou estados externos. Funções puras são mais fáceis de testar, debugar e reutilizar. Quando efeitos colaterais forem necessários, deixe isso explícito no nome da função.

# Executando o Arquivo

Após escrever suas funções em um arquivo `.py`, é hora de executá-lo. Veja como organizar e rodar seu código corretamente.

## Estrutura do arquivo `aula_funcoes.py`

```
def saudacao():  
    print("Olá, mundo!")  
  
def dobro(x):  
    return x * 2  
  
def soma(a, b):  
    return a + b  
  
# Bloco principal de execução  
if __name__ == "__main__":  
    saudacao()  
    print(dobro(7))  
    resultado = soma(4, 6)  
    print(f"Soma: {resultado}")
```

## Rodando no terminal

```
python aula_funcoes.py
```

## Saída esperada:

```
Olá, mundo!  
14  
Soma: 10
```

## O bloco `if __name__ == "__main__"`

Esse padrão garante que o código de teste só rode quando o arquivo é executado **diretamente**, e não quando importado como módulo por outro arquivo. É uma boa prática fundamental em Python para estruturar projetos com múltiplos arquivos.



✓ **Resumo da aula:** definimos funções com `def`, usamos parâmetros e `return`, entendemos escopo local e global, e executamos o arquivo pelo terminal. Na próxima aula: funções com parâmetros padrão e funções recursivas!